Software Processes

Lecture 1b

Agenda

- Individual Programming vs. Big Systems Programming
- Software Engineering: Definition and Importance
- The Software Process
- Software Development Life Cycle

Individual Programming vs. Big Systems Programming

Individual Programming vs. Big Systems Programming

- Lots of people write programs.
  - People in business write spreadsheet programs to simplify their jobs;
  - scientists and engineers write programs to process their experimental data;
  - hobbyists write programs for their own interest and enjoyment.
- However, most software development is a professional activity in which software is developed for business purposes.

Individual Programming vs. Big Systems Programming

- The key distinctions are
  - that professional software is intended for use by someone apart from its developer; and
  - that teams rather than individuals usually develop the software.
  - It is maintained and changed throughout its life.
- Software engineering is intended to support professional software development rather than individual programming.
- It includes techniques that support program specification, design, and evolution, none of which are normally relevant for personal software development.

Software Engineering: Definition and Importance

Software Engineering: Origins

- Software engineering as a discipline exists since the 1960s, when participants of the NATO Software Engineering Conference in 1968 at Garmisch, Germany, recognised that there was a "software crisis" due to the increased complexity of the systems and of the software running (on) these systems.
- The "software crisis" was a term to refer to the problem of building large, reliable software systems in a controlled, cost-effective way.
- The increased complexity of the systems was becoming unmanageable by the hardware developers who were also often the software developers, because they knew best the hardware and there was little knowledge of software development processes, methods, and tools.

Software Engineering: Origins

- The software crisis led to the acknowledgement that software engineering is more than computing theories and efficiency of code and that it requires dedicated research. Thus, this crisis was the starting point of software engineering research.
- Software engineering research acknowledged early that software engineering is fundamentally an empirical discipline, thus further distinguishing computer science from software engineering, because
  - (1) software is immaterial and does not obey physical laws; and
  - (2) software is written by people for people.

Software Engineering: A Brief History

- 1970s Software engineering in the 1970s can be symbolized by the term structure or “structured programming.” Success was made not only in academic research but also in industrial practice.
- 1980s In the 1980s, software engineering went through branching. The study of programming language and programming methods was separated or returned to their originally established position. The remaining body emphasized the aspect of management, particularly managing large-scale system development projects.

Software Engineering: A Brief History

- 1990s The 1990s was the time of object orientation. Smalltalk-80 was introduced in 1980, but the object-oriented design (OOD) and object-oriented analysis (OOA) were highlighted around 1990.
- 2000s In the 2000s, multiple buzzwords appeared and then disappeared or are still surviving, e.g., ubiquitous/pervasive and self-organizing/autonomic. These words represent some aspects of software system structure and behaviour, but in the real world they are more visible as mobile devices, cloud computing, and Internet of Things (IoT)

Software

- Software takes on a dual role. It is a product, and at the same time, the vehicle for delivering a product.
- From the point of view of a software engineer, the work product is the set of programs, content (data), and other work products that are computer software.
- But from the user's viewpoint, the work product is the resultant information that somehow makes the user's world better.
- Software delivers the most important product of our time—information.
- It transforms personal data (e.g., an individual's financial transactions) so that the data can be more useful in a local context.

Defining Software

- Software is: (1) instructions (computer programs) that when executed provide desired features, function, and performance; (2) data structures that enable the programs to adequately manipulate information, and (3) descriptive information in both hard copy and virtual forms that describes the operation and use of the programs.
- A good software should deliver the required functionality and performance to the user and should be maintainable, dependable and usable.

Software Categories

- Two broad categories of software are evolving to dominate the industry:
  - WebApps: Semantic Web technologies (often referred to as Web 3.0) have evolved into sophisticated corporate and consumer applications that encompass semantic databases that provide new functionality that requires Web linking, flexible data representation, and external access APIs.
  - Mobile Applications: In most instances, mobile applications encompass a user interface that takes advantage of the unique interaction mechanisms provided by a mobile platform (e.g., iOS, Android)

Defining Software Engineering

- Engineering is a scientific field and job that involves taking our scientific understanding of the natural world and using it to invent, design, and build things to solve problems and achieve practical goals.
- Software engineering is a systematic approach to the production of software that takes into account practical cost, schedule, and dependability issues, as well as the needs of software customers and producers.

Defining Software Engineering

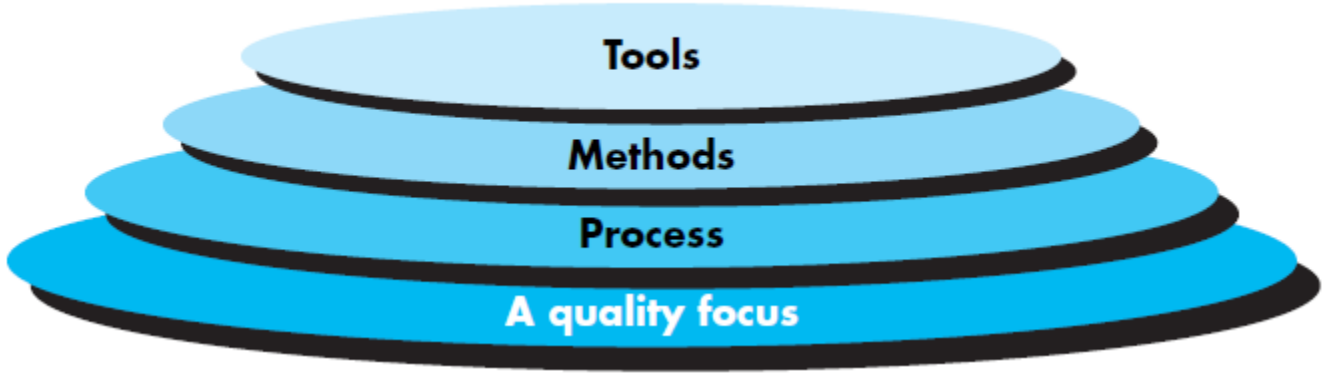
- "Software engineering is the application of a systematic, disciplined, quantifiable approach to the development, operation, and maintenance of software; that is, the application of engineering to software" – SWEBOK.
- "Software engineering is an engineering discipline that is concerned with all aspects of software production from the early stages of system specification through to maintaining the system after it has gone into use" – Sommerville.

Defining Software Engineering

- Software engineering is a layered technology.
  - any engineering approach must rest on an organizational commitment to quality.
  - The foundation for software engineering is the process layer. Process defines a framework that must be established for effective delivery of software engineering technology.

FIGURE 2.1

Software engineering layers



By the way…What Do We Mean by Software Quality?

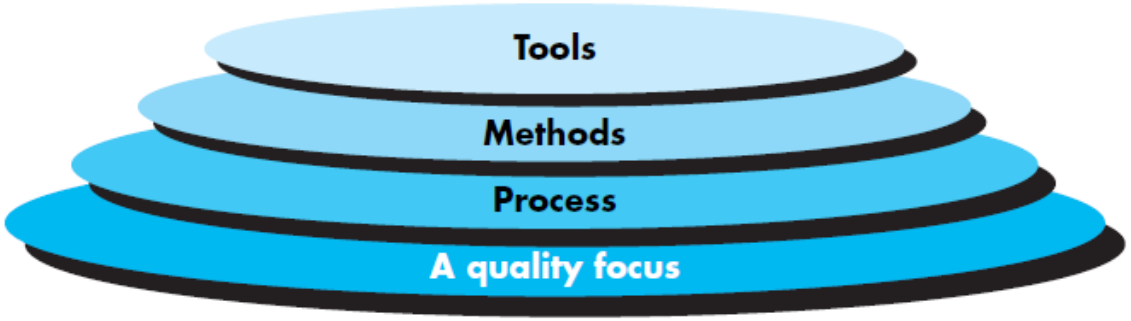
- Software quality is defined as the capability of software product to satisfy stated and implied needs under specified conditions.
- It is also defined as the degree to which a software product meets established requirements;
- however, quality depends upon the degree to which those established requirements accurately represent stakeholder needs, wants, and expectations.

Defining Software Engineering

- Software engineering is a layered technology (continuation).
  - Software engineering methods provide the technical how-to's for building software. Methods encompass a broad array of tasks that include communication, requirements analysis, design modelling, program construction, testing, and support.
  - Software engineering tools provide automated or semi-automated support for the process and the methods.

FIGURE 2.1

Software engineering layers

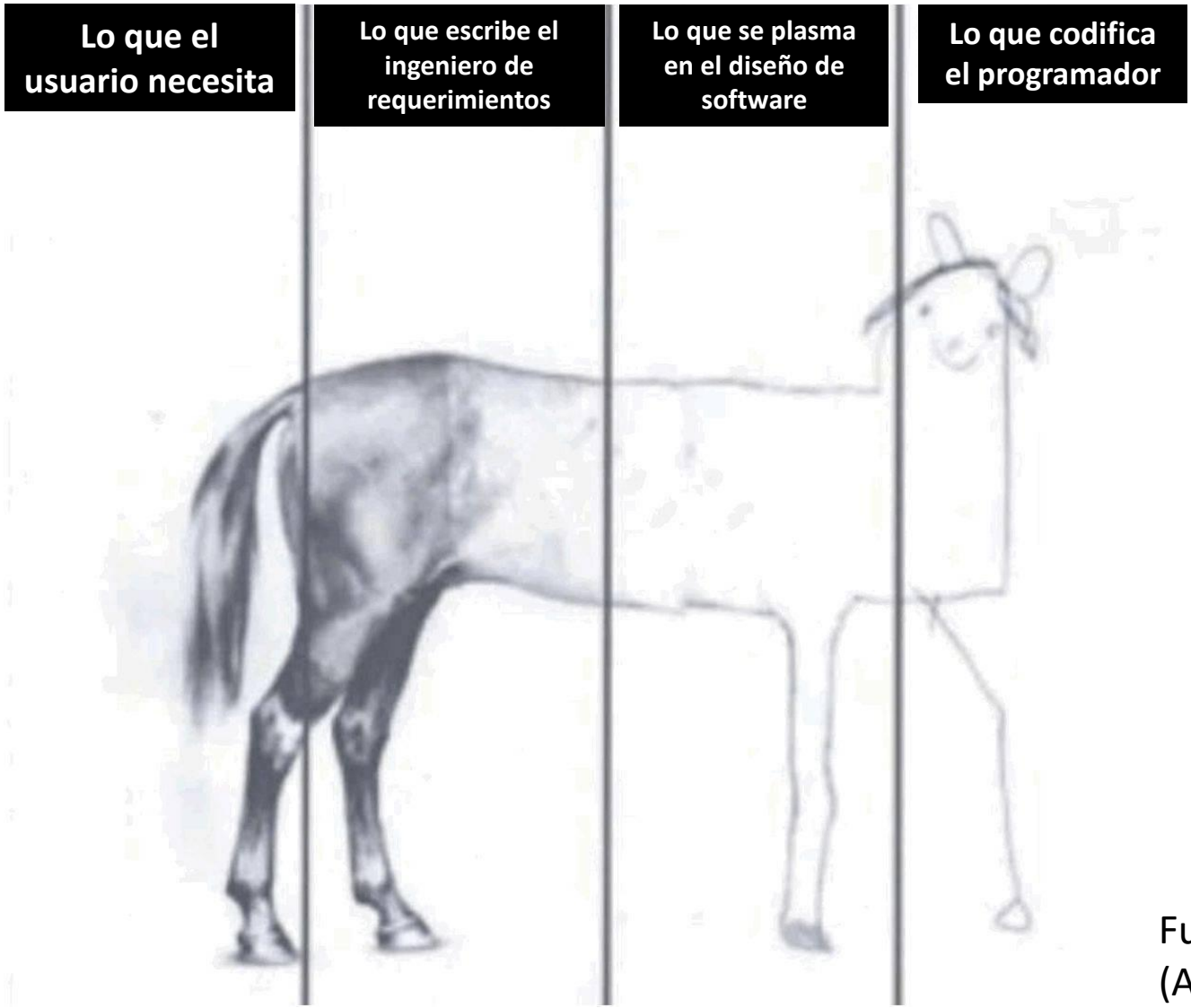


Software Engineering: Importance

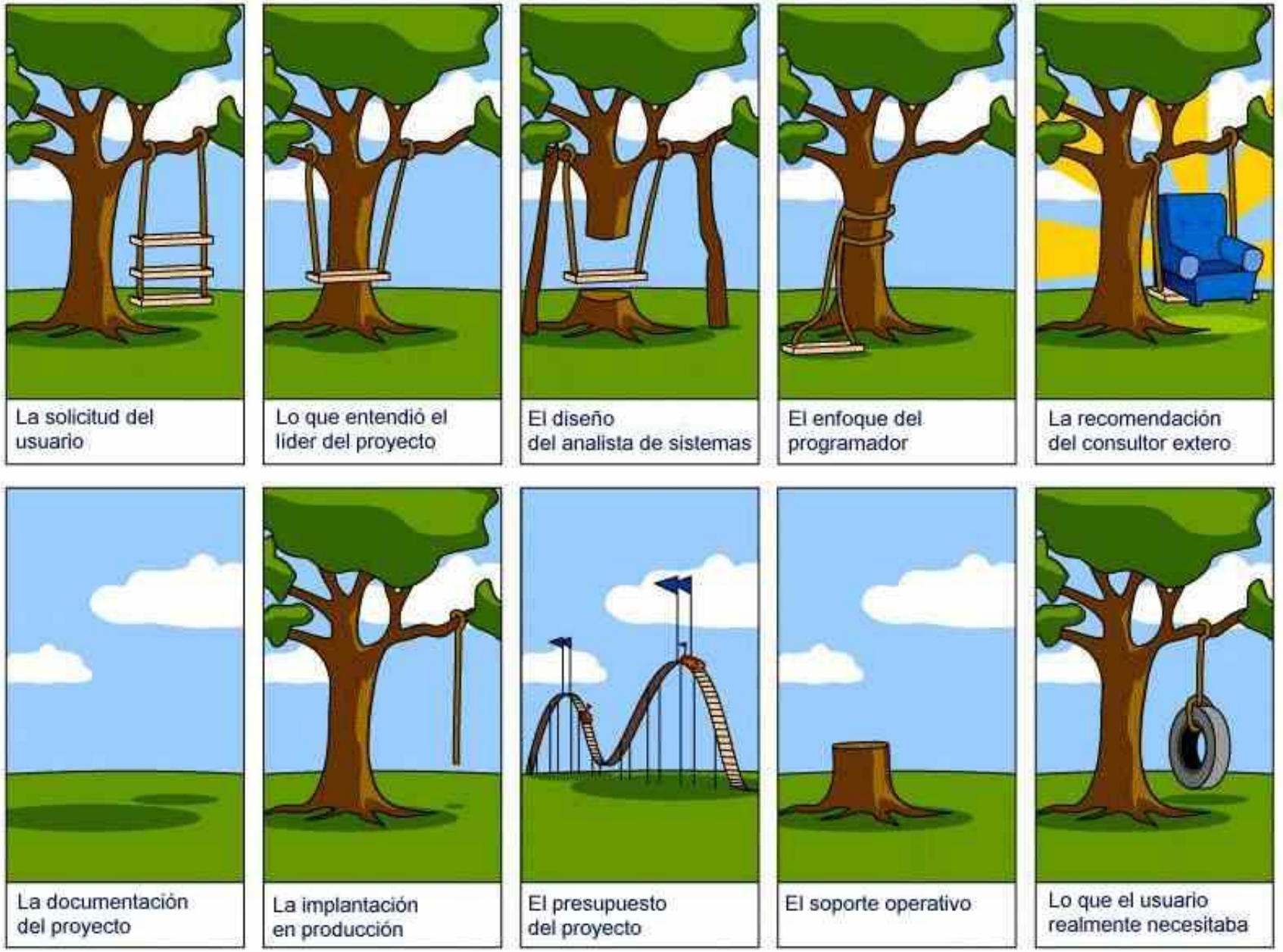
- Software engineering is important because it enables us to build complex systems in a timely manner and with high quality.
- It imposes discipline to work that can become quite chaotic, but it also allows the people who build computer software to adapt their approach in a manner that best suits their needs.

Software Engineering: Importance

- In general, software engineering is important because:
  - More and more, individuals and society rely on advanced software systems. We need to be able to produce reliable and trustworthy systems economically and quickly.
  - It is usually cheaper, in the long run, to use software engineering methods and techniques for professional software systems rather than just write programs as a personal programming project. Failure to use software engineering method leads to higher costs for testing, quality assurance, and long-term maintenance.



Fuente: Dra. Sofía Cabrera (Adaptación)


La solicitud del usuario
Lo que entendió el líder del proyecto
El diseño del analista de sistemas
El enfoque del programador
La recomendación del consultor extero
La documentación del proyecto
La implantación en producción
El presupuesto del proyecto
El soporte operativo
Lo que el usuario realmente necesitaba

The Software Process

The Software Process

- A process is a collection of activities, actions, and tasks that are performed when some work product is to be created.
- An activity strives to achieve a broad objective (e.g., communication with stakeholders) and is applied regardless of the application domain, size of the project, complexity of the effort, or degree of rigor with which software engineering is to be applied.
- An action (e.g., architectural design) encompasses a set of tasks that produce a major work product (e.g., an architectural model).
- A task focuses on a small, but well-defined objective (e.g., conducting a unit test) that produces a tangible outcome.

The Software Process

- The systematic approach that is used in software engineering is sometimes called a software process.
- A software process is a sequence of activities that leads to the production of a software product.
- According to Sommerville, four fundamental activities are common to all software processes.
  1. Software specification, where customers and engineers define the software that is to be produced and the constraints on its operation.
  2. Software development, where the software is designed and programmed.
  3. Software validation, where the software is checked to ensure that it is what the customer requires.
  4. Software evolution, where the software is modified to reflect changing customer and market requirements.

Types of Software

- These generic activities may be organized in different ways and described at different levels of detail, depending on the type of software being developed (e.g., real-time software, e-commerce systems).
  - Stand-alone applications
  - Interactive transaction-based applications
  - Embedded control systems
  - Batch processing systems
  - Entertainment systems
  - Systems for modelling and simulation
  - Data collection and analysis systems
  - Systems of systems
  - Among others

The Software Process

- According to Maxim and Pressman, the generic process framework for software engineering encompasses five activities:
  1. Communication. Before any technical work can commence, it is critically important to communicate and collaborate with the customer (and other stakeholders). The intent is to understand stakeholders' objectives for the project and to gather requirements that help define software features and functions.
  2. Planning. A software project is a complicated journey, and the planning activity creates a "map" that helps guide the team as it makes the journey. The map—called a software project plan—defines the software engineering work by describing the technical tasks to be conducted, the risks that are likely, the resources that will be required, the work products to be produced, and a work schedule.
- A stakeholder is anyone who has a stake in the successful outcome of the project—business managers, end users, software engineers, support people, etc.

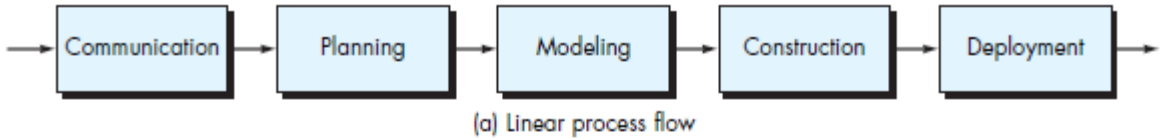
The Software Process

- According to Maxim and Pressman, the generic process framework for software engineering encompasses five activities (continuation):
  3. Modelling. Whether you're a landscaper, a bridge builder, an aeronautical engineer, a carpenter, or an architect, you create a "sketch" of the thing so that you'll understand the big picture—what it will look like architecturally, how the constituent parts fit together, etc. A software engineer does the same thing by creating models to better understand software requirements and the design that will achieve those requirements.
  4. Construction. What you design must be built. This activity combines code generation (either manual or automated) and the testing that is required to uncover errors in the code.
  5. Deployment. The software (as a complete entity or as a partially completed increment) is delivered to the customer who evaluates the delivered product and provides feedback based on the evaluation.

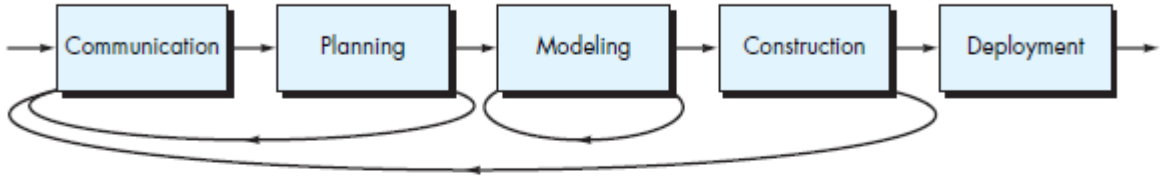
Process Adaptation

- Software engineering process is not a rigid prescription that must be followed dogmatically by a software team.
- Rather, it should be agile and adaptable (to the problem, to the project, to the team, and to the organizational culture).
- Therefore, a process adopted for one project might be significantly different than a process adopted for another project.

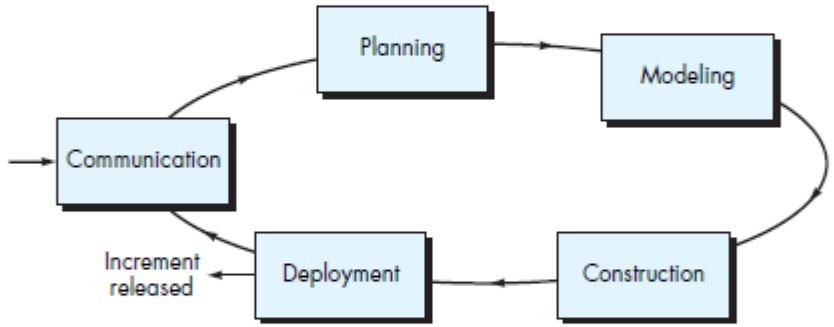
FIGURE 3.2 Process flow



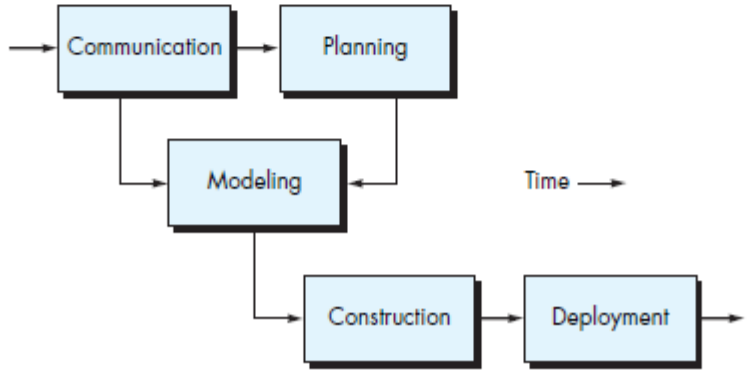
(a) Linear process flow



(b) Iterative process flow



(c) Evolutionary process flow



(d) Parallel process flow

Software Development Life Cycle

Software Development Life Cycle

- A software process model (sometimes called a Software Development Life Cycle or SDLC model) is a simplified representation of a software process.
- Each process model represents a process from a particular perspective and thus only provides partial information about that process.
  - For example, a process activity model shows the activities and their sequence but may not show the roles of the people involved in these activities.

Software Development Life Cycle

- These generic models are high-level, abstract descriptions of software processes that can be used to explain different approaches to software development.
- You can think of them as process frameworks that may be extended and adapted to create more specific software engineering processes.
- Some well-known software process models are:
  - The Waterfall Model
  - The Incremental Model
  - Evolutionary Process Models
  - Concurrent Models

Software Development Life Cycle

- There are also specialized process models which take on many of the characteristics of one or more of the traditional models presented in the preceding slide.
  - Component-Based Development
  - Aspect-Oriented Software Development
  - Rational Unified Process (RUP) Model
- We will see them all in detail the next lecture.

Next On

Take Away Points

- Professional software development
  - How does it differ from individual programming?
- Software engineering?
  - What is it?
  - Why is it important?
- The software process
  - What is it?
  - Which are its basic building blocks?
  - What is a software process model?

Further Reading

- Cha, Taylor, and Kang, “Handbook of Software Engineering”
  - Chapter: Empirical Software Engineering
  - Chapter: Key Software Engineering Paradigms and Modelling Methods
- Pressman and Maxim, “Software Engineering”
  - Chapter 1: The Nature of Software
  - Chapter 2: Software Engineering
- Ian Somerville, “Software Engineering”
  - Chapter 1: Introduction to Software Engineering
  - Chapter 2: Software Processes
- Bourque and Fairley, “SWEBOK: Software Engineering Body of Knowledge”
  - Version 3

Next Lecture

- Software Process Models